

Project 1.4 Crazyflie Lab

March 26, 2025

Ethan Senatore

Vineet Pasumarti

Allen Zhou

Raymond Yang

I. SYSTEM OVERVIEW

The objective of this lab was to deploy a quadrotor controller, waypoint planner, and trajectory generator on real hardware.

In our first lab session, we implemented both a controller and waypoint planner on a CrazyFlie 2.0. We first validated the quadrotor's stability with a hover, then tuned our controller's z position gain using an altitude step response. We then tuned x and y position gains using a lateral step response. We evaluated our controller against a test "box" maneuver, and with our extra lab time we wrote our own test waypoints to fly the drone in a rectangular spiral and land.

In our second lab session, we deployed our stack on the CrazyFlie to plan a path through a maze and execute the resulting trajectory. This required us to re-evaluate our gains and fine-tune the margins of our occupancy map to ensure the safe traversal of the drone from start to finish. Following both sessions, we were able to demonstrate successful control and flight path generation. Tuning parameters on real hardware allowed us to be less risk-averse and execute more aggressive trajectories that navigate the cluttered environment faster and smoother.

The CrazyFlie 2.0 is a palm-sized quadrotor that we cover with markers to be tracked by a VICON system. VICON is a motion capture system that uses multiple infrared cameras to track the position and orientation of an object. The quadrotor's position and orientation are tracked by the VICON and the angular velocity and acceleration from the onboard accelerometer and gyroscope are all radio transmitted to the lab computers. This information is fed to our controller as an input, which runs on the lab computers to generate a desired state and the necessary motor commands are then transmitted back to the quadrotor.

II. METHODS

The controller we elected to use for our experiments was a geometric nonlinear controller. In an intuitive sense, the controller is structured around the idea of orienting the quadrotor's z -axis in the direction we desire to move to. This controller allows for aggressive maneuvers and requires fine-tuning of both the damping and gain. Using the Project 1.1 handout as a reference, we will define the theoretical framework of the controller. We start with the PD controller equation:

$$\ddot{r}^{des} = \ddot{r}_T - K_d(\dot{r} - \dot{r}_T) - K_p(r - r_T)$$

To compute the inputs u_1 and u_2 in order to generate our desired acceleration we use the following equation.

$$m\ddot{r}^{des} + \begin{bmatrix} 0 \\ 0 \\ mg \end{bmatrix} = u_1 R \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

We can identify that the left hand side of the equation is simply the desired force combined with the gravitational force. Thus, we can simplify the equation in order to solve for u_1 as follows:

$$F^{des} = m\ddot{r}^{des} + \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

$$u_1 = R \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} F^{des}$$

Calculating u_2 requires a slightly more complex process. Based off of the previously laid out intuition, we seek to align the thrust vector with the quadrotor's z -axis. Thus,

$$c_3^{des} = \frac{F^{des}}{\|F^{des}\|}$$

Next, we seek to align the quadrotor's y -axis with the desired yaw direction while maintaining the orthogonality between the y and x -axes. To do so, we first define the following term:

$$a_\psi = \begin{bmatrix} \cos\psi_T \\ \sin\psi_T \\ 0 \end{bmatrix}$$

this vector represents the yaw direction in the world

x-y plane. Next, we can calculate the y-axis vector as follows:

$$c_2^{des} = \frac{c_3^{des} \times a_\psi}{\|c_3^{des} \times a_\psi\|}$$

Finally, we can define the desired R^{des}

$$R^{des} = [c_2^{des} \times c_3^{des}, c_2^{des}, c_3^{des}]$$

This rotation matrix represents the desired orientation we wish to achieve at a given time step. The next step is to measure the error between our current and desired orientation.

$$e_R = \frac{1}{2}(R^{desT}R - R^TR^{des})^\vee$$

Another aspect of control we may care about is the difference between our desired and current angular velocities. We can easily define this term as:

$$e_\omega = \omega - \omega^{des}$$

Finally, we can define u_2 as:

$$u_2 = I(-K_R e_R - K_\omega e_\omega)$$

Using this controller, we have four tunable gains that we had to first meticulously tune in simulation and then finalize in our experiments. After tuning our controller, in simulation, lab 1, and lab 2 we settled on the following configuration. The 1x3 vector listed in Table I represents the diagonal of a 3x3 matrix.

Gain	Value	Units
K_p	[2.45, 2.45, 10.5]	$1/s^2$
K_d	[5.25, 5.25, 6.3]	$1/s$
K_R	[1750, 1750, 140]	$1/s^2$
K_ω	[175, 175, 105]	$1/s$

TABLE I: FINALIZED GAIN VALUES AFTER TUNING

Intuitively, we interpreted the gains as ways to answer four different questions. Starting with K_p , we viewed this metric as measuring ‘how important is it to achieve a desired position?’. K_d is the measure of ‘how fast do you want to get to the desired position?’. Similarly, we can say that K_R measures ‘how important is it to achieve a desired orientation?’ and K_ω measures ‘how fast do you want to achieve the desired orientation?’. This intuition provided us a steady foundation to work off of and fine tune our parameters. It was fairly easy to tinker with each gain to reach our desired performance. However, we immediately noticed that testing on hardware required us to scale down our gains by around 30%. Additional fine-

tuning during lab 1 and 2 narrowed us down to our final parameters. We can attribute the differences in gains due to model mismatch, lack of simulation fidelity, and flaws in our controller implementation.

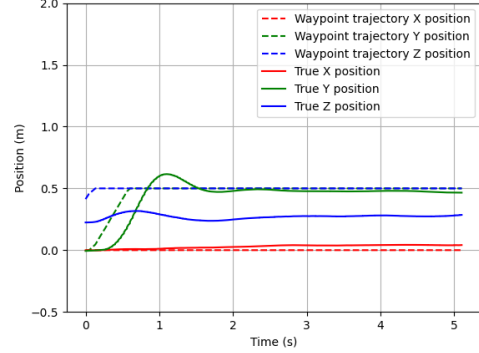


FIGURE I: STEP TUNING TEST IN Y DIRECTION

Figure I shows an experiment we ran to tune our y-direction gain and damping parameters. Based on the plots, we can determine that our controller demonstrates a settling time of around 2 s, a rise time of around 1.1 s, a steady state error of around 2%, and a damping ratio of around 0.4. This experiment revealed a few flaws in our controller. First and foremost, the controller is under-damped and takes fairly long to settle into a steady state. Secondly, we immediately recognized a glaring issue in our z-direction tuning. At first, we attributed the discrepancy in desired and actual position as model mismatch, but in lab 2 we retuned our z gains to fix this issue. The process of fine-tuning the gains was straightforward as we would conduct a test, adjust the parameters, and repeat until we were satisfied.

III. TRAJECTORY GENERATION

In this section, we discuss the approach for the generation of a piecewise trajectory connecting the waypoints from start to target for an input map in the discretized occupancy grid, as well as a method that generates an optimized flight path that aligns with the quadrotor’s maneuverability while maintaining collision-free.

Starting with the default `occupancy_map` file where a 3D occupancy map is generated from the given map environment by splitting the space into a grid of voxels at a given axial resolution, the occupancy map consists of voxels that are marked ‘‘occupied’’ when they intersect an obstacle or hit the boundary within the workspace, and others representing the discretized waypoints that are open access to the graph search planner.

Then, the `graph_search` treats each unoccupied voxel from the previous map as a node in a graph, which is able to develop up to 26 edges connecting its neighbors in 3D. During the lab execution, an A* graph search algorithm is preferred over the plain Dijkstra method, since it in-

incorporates a heuristic measure, which is the straight-line distance to the goal, to guide the search more efficiently. This heuristic attracts the exploration toward the target node, significantly reducing the exploration time to find an optimal path compared to Dijkstra, which blindly develops its path outward. To conclude, the cost of moving between neighboring nodes is the Euclidean distance [l_2 norm], while an admissible heuristic $h(n)$ estimates the cost from the current node to the goal.

$$c(\text{current}, \text{neighbor}) = \|\text{neighbor} - \text{current}\|,$$

$$h(n) = \|n - \text{goal}\|.$$

The total cost at each node is therefore:

$$f(n) = g(n) + h(n),$$

where $g(n)$ represents the cost so far from the start node to the current.

After running A*, we apply the Ramer-Douglas-Peucker (RDP) algorithm to reduce redundant points along nearly straight segments. An RDP tolerance of $\varepsilon = 0.1$ m is used, which prunes interior waypoints whose perpendicular distance to the line between the first and last point in a subset is below ε . The outcome is a sparser set of critical waypoints, preserving the overall path shape but removing unnecessary intermediate points that might complicate maneuvering along the trajectory. We construct a piecewise linear position trajectory to connect along each optimized waypoint, commanding the quadrotor to operate from start to goal. The position where each single segment ends is:

$$\mathbf{x}(t) = \mathbf{p}_i + (\mathbf{v}_i)(t - t_{\text{start},i}), \quad t \in [t_{\text{start},i}, t_{\text{start},i} + T_i],$$

where $\mathbf{v}_i$ is a constant velocity vector for segment i , defined by

$$\mathbf{v}_i = v_i \frac{\mathbf{p}_{i+1} - \mathbf{p}_i}{\|\mathbf{p}_{i+1} - \mathbf{p}_i\|}.$$

Hence, velocity is constant in each segment, and the position changes linearly from $\mathbf{p}_i$ to $\mathbf{p}_{i+1}$. Once t exceeds $t_{\text{start},i} + T_i$, we switch to the next segment $i + 1$ until the Crazyflie hits the target. In our code, v_i is chosen based on the segment length and the angular change between consecutive segments (with mild speed reductions if there is a sharp turn). The total trajectory time becomes

$$T_{\text{total}} = \sum_{i=0}^{N-1} T_i.$$

where a constant speed v_i for each segment $[\mathbf{p}_i \rightarrow \mathbf{p}_{i+1}]$ is:

$$T_i = \frac{\|\mathbf{p}_{i+1} - \mathbf{p}_i\|}{v_i}.$$

Since each segment uses constant velocity, the overall trajectory is only continuous in position (C^0 -continuous); velocity changes instantaneously at segment boundaries. However, we limit the speed of the Crazyflie to under 1

m/s, which prevents aggressive maneuvering during the tests. By including an obstacle inflation margin of 0.2 m and a resolution of the occupancy grid of $\Delta x = \Delta y = \Delta z = 0.25$ m, we ensure that the resulting waypoints remain safely away from obstacles while keeping a low computational load. Hence, while the piecewise-linear approach does not strictly enforce higher-order continuity, it is straightforward to implement, fast to compute, and guarantees collision-free given the flight map. The plots below show (1) discrete waypoints and path, (2) 3D quadrotor trajectory, and (3) position, velocity, and acceleration log in the map `maze_2025_1`.

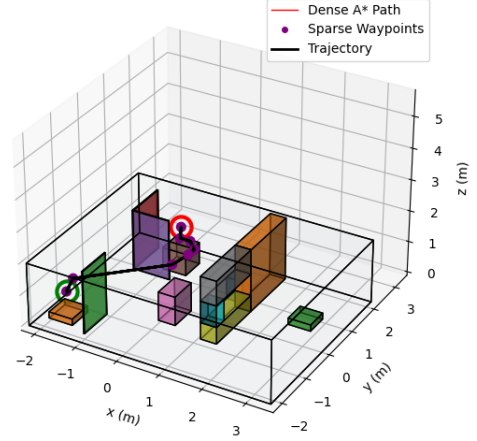


FIGURE II: DISCRETE WAYPOINTS AND PATH

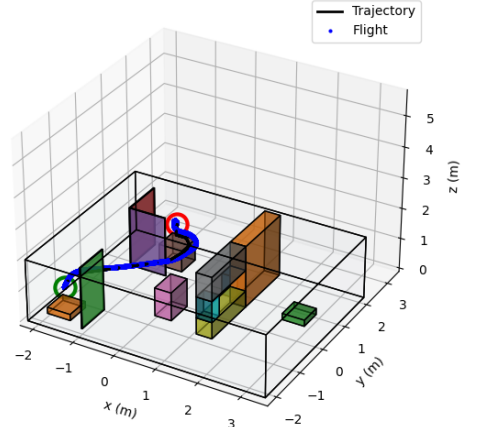


FIGURE III: 3D QUADROTOR TRAJECTORY

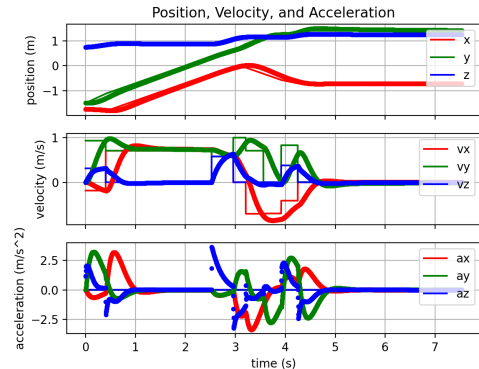


FIGURE IV: POSITION, VELOCITY, AND ACCELERATION LOG

IV. MAZE FLIGHT EXPERIMENTS

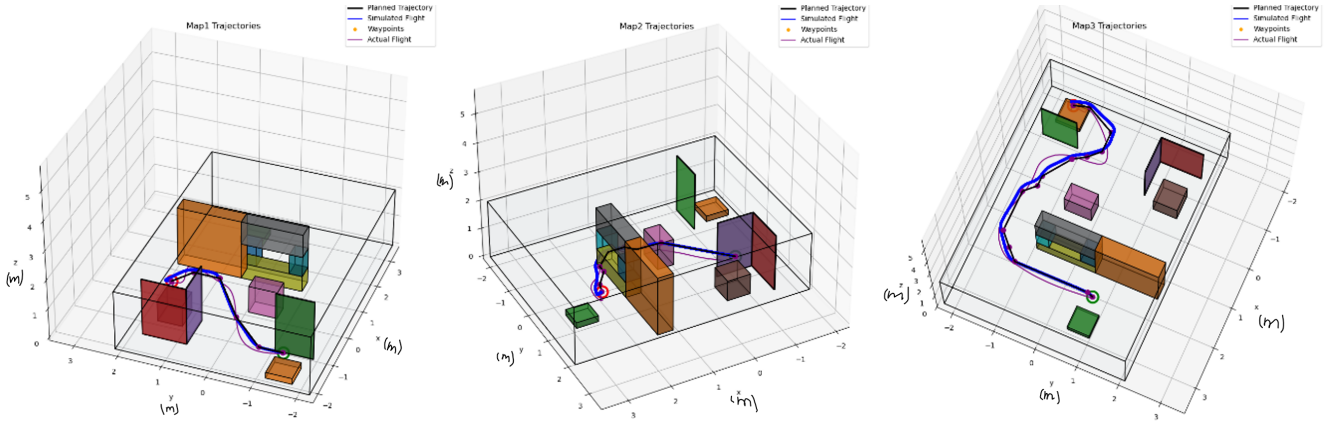


FIGURE V: : MAZE 1-3 (LEFT TO RIGHT) WITH WORLD OBSTACLE, WAYPOINTS, PLANNED TRAJECTORIES, AND ACTUAL FLIGHT PATH

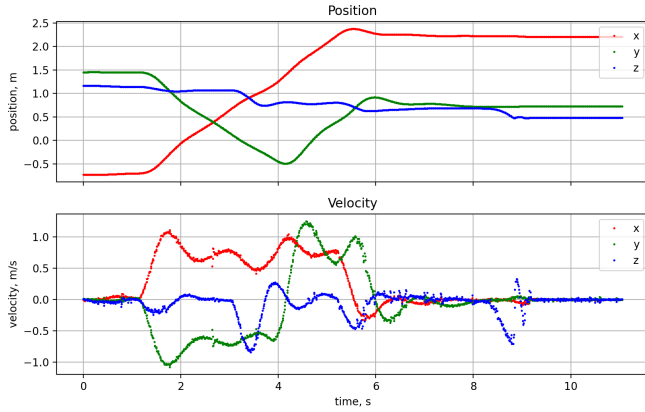


FIGURE VI: : MAP 2 ACTUAL POSITION AND VELOCITY

We observe discrepancies between the planned trajectory and actual flight path in all three maps, especially during sharp turns and maneuvers around obstacles. One possible cause for the tracking error comes from the motor thrust limitations. The maximum thrust capabilities of the flight motors restricted the drone's response to control inputs during high-demand maneuvers, leading to larger than anticipated deviations from the planned path. In addition, interruptions in positional data due to occlusions of VICON cameras by obstacles resulted in inaccurate state estimations, causing the drone to stray off course. The discrepancies between the simulated and actual weight of the drone can be another cause, it led to inadequate PID controller settings and impacting the drone's dynamic response to control commands. To achieve safe trajectories, we can directly adjust the trajectory planning algorithm to include larger safety margins around obstacles, accounting for potential hardware limitation during path planning. We can also further adjust PID parameters using actual

drone weight data.

Since we have already observed undesired flight paths, it can be dangerous to make the trajectory more aggressive. The limited maximum thrust of the flight motors restricted the drone's ability to execute rapid or steep maneuvers, essential for more aggressive trajectories. The observed inaccuracies in positional data due to VICON camera occlusions and the lack of precise PID tuning for the actual drone weight meant that increasing the aggressiveness of the trajectories could lead to higher risks of collision or loss of control.

To improve the system reliability and speed, one change we can make is tune the PID using real world data to reduce the discrepancy between simulated trajectory and actual pathways. A more robust trajectory planning algorithm can also improve the system reliability and speed, with higher dimension spline fitting, the trajectory can be smooth and reduce the potential issue with motor torque limitations. Incorporate higher resolution in path planning is another effective approach to enhance both the speed and reliability of the drone system. This allows the trajectory to be optimized at a more detailed level, enabling smoother transitions and tighter navigation around obstacles.

If we had one more session in the lab to do something interesting, it will be interesting to try implementing more advanced control techniques with PID controller parameters dynamically based on observed performance metrics and environmental conditions encountered during flight. The goal would be to see how well the drone can optimize its flight stability and accuracy in response to dynamically changing conditions within the maze, potentially leading to more reliable and efficient navigation.